DISEÑO WEB EN ENTORNO CLIENTE · 2.º DAW

Comunicación asíncrona cliente-servidor

UT 5.1 · Errores, JSON y programación asíncrona

Errores | JSON | Copiar objetos | Asincronía

PLAN DEL TEMA

Cuatro pasos, de lo sencillo a lo complejo

1

Errores

Qué pasa cuando algo falla y cómo controlarlo con try/catch.

2

JSON

El formato en el que viajan los datos: para qué sirve, stringify y parse.

3

Copiar objetos

Por qué igualar no copia, y cómo hacer copias de verdad.

4

Asincronía

Esperar sin bloquear: setTimeout, promesas y async/await.

1

Gestión de errores

try, catch, finally y throw

ERRORES

Cuando algo falla, el programa se detiene

1. Se produce un error
Una variable que no existe, un dato mal formado…

2. JavaScript lanza una excepción
Un objeto Error con un nombre y un mensaje.

3. Si nadie la controla…
El programa se para en esa línea y el resto no se ejecuta.

```
console.log('Antes');
console.log(variableInexistente);
// Aquí se produce el error
console.log('Después');
// Nunca se llega a esta línea
```

```
Antes
ReferenceError: variableInexistente is not defined
```

ERRORES

Los cuatro errores que verás con más frecuencia

ReferenceError

Usas una variable que no existe.

```
console.log(edad);
```

TypeError

Haces con un valor algo que su tipo no permite.

```
const alumno =
null;
alumno.nombre;
```

SyntaxError

El texto está mal escrito. Muy típico con JSON.

```
JSON.parse('{nombre
: 1}');
```

RangeError

Un número está fuera del rango permitido.

```
new Array(-1);
```

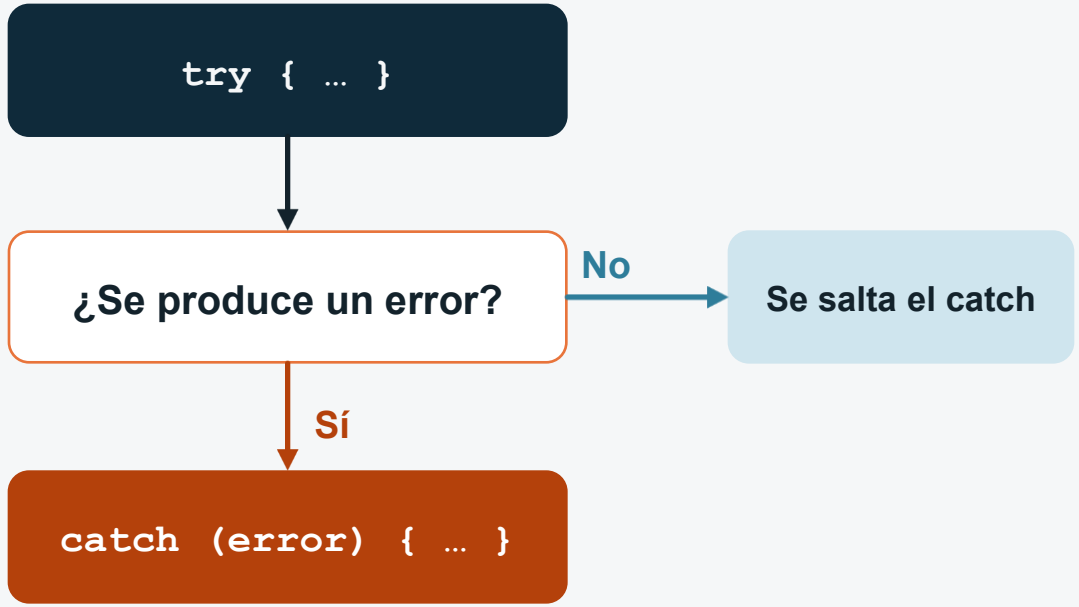
ERRORES

try / catch: intenta y, si falla, reacciona

```
try {
  console.log('Intento…');
  console.log(variableInexistente);
  console.log('No llego aquí');
} catch (error) {
  console.log('Error controlado:', error.name);

}
console.log('El programa sigue');
```

```
Intento…
Error controlado: ReferenceError
El programa sigue
```



ERRORES

finally: se ejecuta siempre

```
try {
  console.log('Abro el archivo');
  console.log(variableInexistente);
} catch (error) {
  console.log('Error:', error.name);
} finally {
  console.log('Cierro el archivo');
}
```

```
Abro el archivo
Error: ReferenceError
Cierro el archivo
```

try
Lo que quieres intentar.

catch
Qué hacer solo si hay error.

finally
Qué hacer siempre: haya ido bien o mal. Ideal para «limpiar» (cerrar, ocultar el cargando…).

ERRORES

throw: lanzar tu propio error

```
function comprobarEdad(edad) {
  if (edad < 0 || edad > 120) {
    throw new RangeError('Edad fuera de rango');
  }
  return edad;
}

try {
  comprobarEdad(200);
} catch (error) {
  console.log(error.name, '-', error.message);
}
```

```
RangeError - Edad fuera de rango
```

Tú decides cuándo es un error

Un dato puede ser «válido» para JavaScript y no serlo para tu programa.

throw detiene la función

Igual que un error del sistema: lo que sigue no se ejecuta.

El catch lo recoge

Con el mismo name y message que cualquier otro error.

ERRORES

¿Cuándo usar try/catch?

Úsalo cuando…

Conviertes texto externo con **JSON.parse**

Pides datos a un servidor (lo verás en 5.2)

Procesas lo que escribe el usuario

Lees datos que no controlas: archivos, almacenamiento…

Evítalo cuando…

Envuelves todo el programa «por si acaso»

Un simple **if** resuelve la comprobación

El fallo es un despiste tuyo: mejor corregir el código

El catch queda vacío: un error silenciado es muy difícil de encontrar

Regla: usa try/catch solo para lo que puede fallar por causas ajenas a tu código.

2

JSON

El formato en el que viajan los datos

JSON

¿Por qué existe JSON?

Navegador · JavaScript

```
const libro = {
  titulo: 'Dune',
  anio: 1965
};
```

Un objeto de JavaScript

stringify

Texto JSON

```
{"titulo":"Dune",
 "anio":1965}
```

Un texto: esto sí puede viajar

por la red

Servidor

Java, PHP, Python, C#…

Lee el texto y lo convierte en sus propios objetos

Por la red solo viajan textos. JSON es un texto con reglas que entienden todos los lenguajes.

JSON

¿Y qué es una API REST?

API

Un servicio al que tu página puede pedir datos, como un mostrador al que haces un pedido.

REST

Una forma ordenada de organizarlas: cada dato tiene su dirección, por ejemplo **/libros** o **/libros/1**.

La respuesta

Casi siempre llega en formato JSON. Por eso hay que dominarlo.

Respuesta de la API a «dame los libros»:

```
[
  { "id": 1,
    "titulo": "El nombre de la rosa" },
  { "id": 2,
    "titulo": "Cien años de soledad" }
]
```

JSON

JSON: JavaScript Object Notation

Es solo texto

Un archivo .json o una respuesta del servidor: nada más que caracteres.

Es universal

Nació de los objetos de JavaScript, pero hoy lo usan todos los lenguajes.

Se parece a un objeto… pero no lo es

Tiene reglas más estrictas, que veremos enseguida.

Un ejemplo de documento JSON:

```
{
  "nombre": "Ana",
  "edad": 19,
  "aprobada": true,
  "notas": [7, 8.5],
  "tutor": null
}
```

JSON

Seis tipos de valor, nada más

Texto

`"Ana"`

Siempre entre comillas dobles

Número

`19 · 8.5`

Sin comillas, con punto decimal

Booleano

`true · false`

En minúsculas y sin comillas

Nulo

`null`

«No hay valor»

Array

`[7, 8.5, 10]`

Lista ordenada de valores

Objeto

`{ "id": 1 }`

Pares clave: valor

JSON

Cuatro reglas que hay que cumplir

1. Claves con comillas dobles

✗ `{ nombre: "Ana" }`

✓ `{ "nombre": "Ana" }`

2. Textos con comillas dobles

✗ `{ "nombre": 'Ana' }`

✓ `{ "nombre": "Ana" }`

3. Sin coma al final

✗ `{ "edad": 19, }`

✓ `{ "edad": 19 }`

4. Sin comentarios

✗ `{ "edad": 19 // años }`

✓ `{ "edad": 19 }`

JSON

JSON no es un objeto de JavaScript

	Objeto literal de JS	JSON
Qué es	Un valor en la memoria	Un texto
Claves	Con o sin comillas	Siempre comillas dobles
Textos	Comillas simples o dobles	Solo comillas dobles
Coma final y comentarios	Permitidos	Prohibidos
Funciones y undefined	Permitidos	No existen

JSON.stringify: de objeto a texto

```
const libro = { titulo: 'Dune', anio: 1965 };
const texto = JSON.stringify(libro);

console.log(texto);
console.log(typeof texto);
```

```
{"titulo":"Dune","anio":1965}
string
```

Entra
Un objeto, un array, un número…

`JSON.stringify( )`

Sale
Un texto en formato JSON

stringify: dos opciones útiles

```
const libro = { titulo: 'Dune', autor: 'Frank Herbert', anio: 1965 };
```

1. Con sangría (legible)

```
JSON.stringify(libro, null, 2)
```

```
{
  "titulo": "Dune",
  "autor": "Frank Herbert",
  "anio": 1965
}
```

2. Solo algunas propiedades

```
JSON.stringify(libro, ['titulo', 'anio'])
```

```
{"titulo":"Dune","anio":1965}
```

Útil para no enviar datos que no hacen falta. El autor se queda fuera.

JSON

JSON.parse: de texto a objeto (con try/catch)

```
function leerLibro(texto) {
  try {
    const libro = JSON.parse(texto);
    console.log(libro.titulo);
  } catch (error) {
    console.log('JSON no válido:', error.name);
  }
}
```

```
leerLibro('{"titulo":"Dune"}');
leerLibro('{titulo: Dune}');
```

```
Dune
JSON no válido: SyntaxError
```

Entra
Un texto en formato JSON

`JSON.parse( )`

Sale
Un valor de JavaScript

Si el texto no es JSON válido, lanza SyntaxError: usa siempre try/catch.

JSON

Lo que se pierde por el camino

```
const datos = {
  nombre: 'Ana',
  apodo: undefined, // se omite
  saludar() { return 'hola'; }, // se omite
  nota: NaN, // pasa a null
  registro: new Date('2026-01-15T10:00:00Z'),
};
console.log(JSON.stringify(datos));
```

```
{"nombre":"Ana","nota":null,"registro":"2026-01-15T10:00:00.000Z"}
```

undefined y funciones
Desaparecen.

NaN
Se convierte en null.

Date
Pasa a texto, y con parse vuelve como texto, no como fecha.

Lo que va y vuelve no siempre es idéntico.

JSON

Importar un archivo JSON como módulo

```
import libros from './datos/libros.json' with { type: 'json' };

console.log(`Hay ${libros.length} libros en el archivo:`);
for (const libro of libros) {
  console.log(`- ${libro.titulo} (${libro.anio})`);
}
```

```
Hay 3 libros en el archivo:
- El nombre de la rosa (1980)
- Cien años de soledad (1967)
- Fahrenheit 451 (1953)
```

Sin JSON.parse

El archivo llega ya convertido en objeto o array.

La parte nueva

with { type: 'json' } avisa de que el módulo es JSON, no código.

Requisitos

Node.js 22 o superior. En el navegador, el módulo se pide a un servidor: no funciona abriendo el archivo con file://.

JSON

No te fíes de los datos externos

```
const esLibroValido = (libro) => {
  if (typeof libro !== 'object' || libro === null)
    return false;
  if (typeof libro.titulo !== 'string' || libro.titulo ===
'')
    return false;
  if (typeof libro.anio !== 'number') return false;
  return true;
};
```

Que sea JSON válido no garantiza que los datos sean correctos.

```
{"titulo":"Dune","anio":1965}
```
✓ aceptado

```
{"titulo":"Dune","anio":"1965"}
```
✗ el año es un texto

```
{"titulo":"","anio":1965}
```
✗ título vacío

```
{"titulo":"Dune","anio":1965
```
✗ JSON roto: falta la llave final

```
null
```
✗ JSON válido, pero no es un objeto

3

Copiar objetos

Igualar no es copiar

COPIAR OBJETOS

Igualar no copia un objeto

Números y textos: se copia el valor

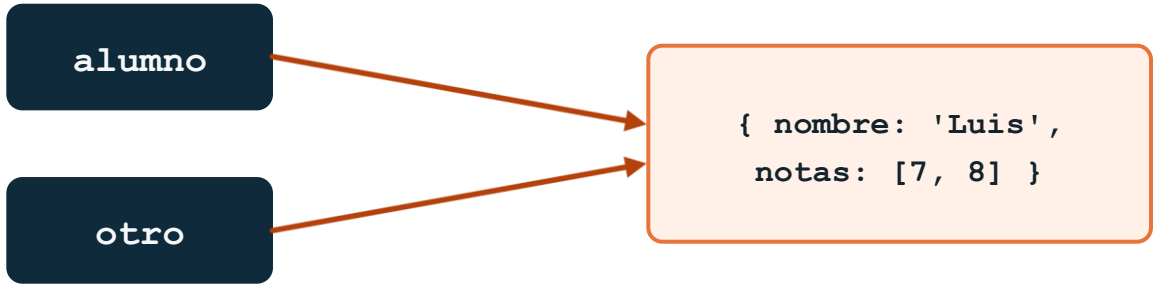
```
let a = 5;
let b = a;
b = 10;
console.log(a, b); // 5 10
```

a = 5

b = 10

Cada variable tiene su propia caja. Son independientes.

Objetos: las dos variables señalan al mismo objeto




```
const otro = alumno;
otro.nombre = 'Luis';
console.log(alumno.nombre); // Luis: ¡también ha cambiado!
```

COPIAR OBJETOS

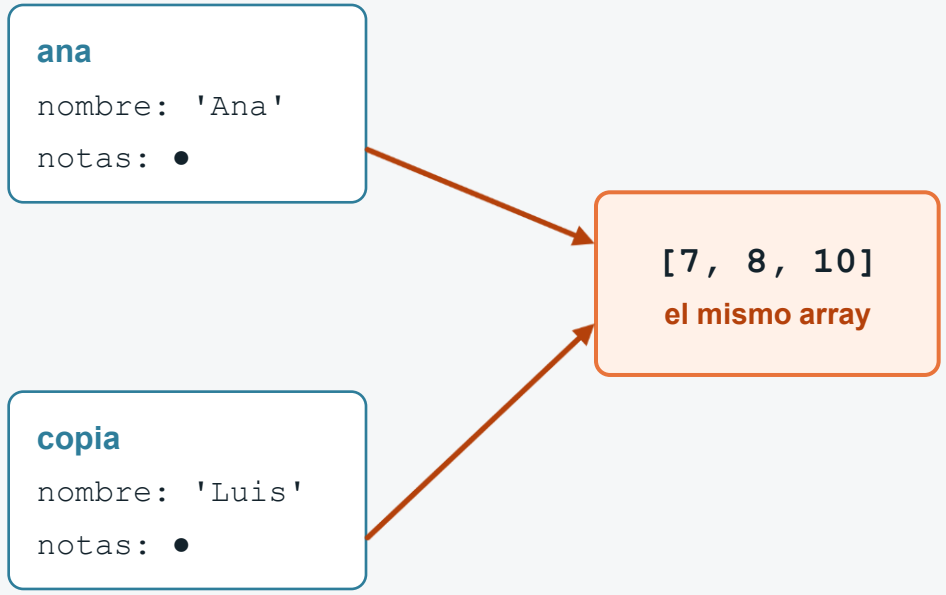
Copia superficial con el operador …

```
const ana = { nombre: 'Ana', notas: [7, 8] };
const copia = { ...ana };

copia.nombre = 'Luis';
console.log(ana.nombre, copia.nombre);

copia.notas.push(10);
console.log(ana.notas);
```

```
Ana Luis
[ 7, 8, 10 ]
```



Lo simple se copia; lo que hay dentro (arrays, objetos) se comparte.

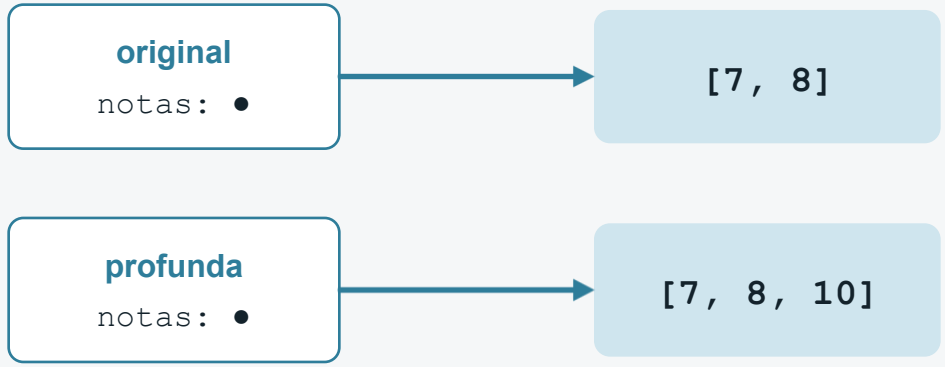
COPIAR OBJETOS

Copia profunda con structuredClone

```
const original = { nombre: 'Ana', notas: [7, 8] };
const profunda = structuredClone(original);

profunda.notas.push(10);
console.log(original.notas, profunda.notas);
console.log(original.notas === profunda.notas);
```

```
[ 7, 8 ] [ 7, 8, 10 ]
false
```



Ojo: no sabe copiar funciones

```
structuredClone({ saludar() {} });
// DataCloneError
```

COPIAR OBJETOS

Resumen: ¿qué uso para copiar?

Forma	¿Qué hace?	Lo de dentro
`otro = alumno`	No copia: mismo objeto	**Todo compartido**
`{ ...alumno }`	Copia superficial	**Arrays y objetos interiores compartidos**
`structuredClone(alumno)`	Copia profunda	**Todo independiente (sin funciones)**

En código antiguo verás el truco **JSON.parse(JSON.stringify(objeto))**. Funciona, pero pierde fechas, undefined y funciones: hoy se usa structuredClone.

4

Programación asíncrona

setTimeout, promesas y async/await

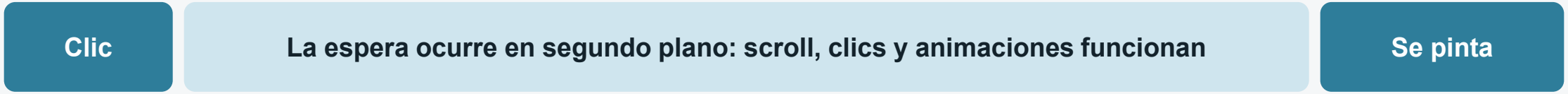
ASINCRONÍA

Esperar sin quedarse parado

Código síncrono: todo espera



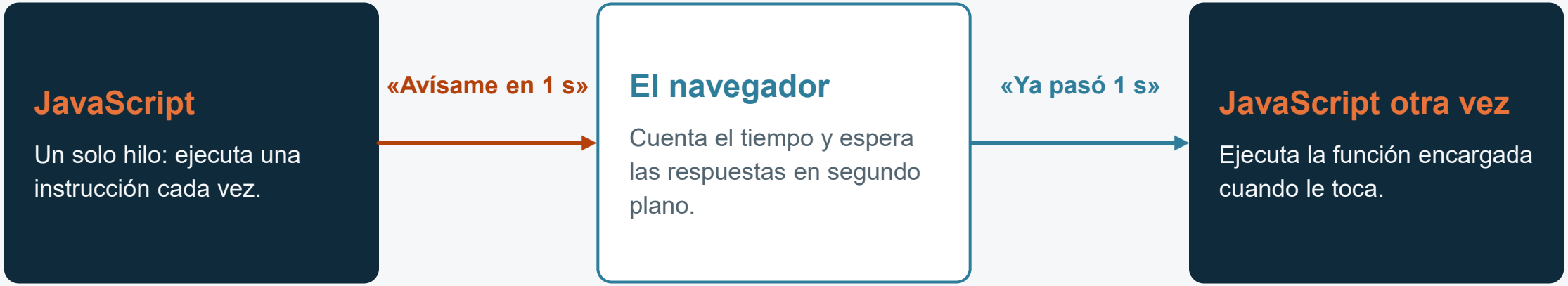
Código asíncrono: la página sigue viva



Pedir datos a un servidor puede tardar segundos. Si el código esperase parado, la página quedaría congelada.

ASINCRONÍA

JavaScript hace una cosa cada vez; el navegador espera por él



Como poner el horno a 30 minutos: no te quedas mirándolo; haces otras cosas y, cuando suena, atiendes.

ASINCRONÍA

setTimeout: ejecutar algo más tarde

```
console.log('A: empiezo');

setTimeout(() => {
  console.log('B: han pasado 1000 ms');
}, 1000);

console.log('C: sigo trabajando');
```

```
A: empiezo
C: sigo trabajando
B: han pasado 1000 ms ← 1 s después
```

`setTimeout(función, ms)`
1.º la función que se ejecutará más tarde; 2.º cuántos milisegundos esperar.

No detiene el programa
Programa el aviso y sigue: por eso C sale antes que B.

Sirve para simular un servidor lento
Con él fabricaremos las esperas de los ejemplos.

ASINCRONÍA

¿En qué orden se ejecuta?

```
console.log('A');
setTimeout(() => console.log('D'), 0);
console.log('C');
```

```
A
C
D
```

Incluso con 0 ms, el setTimeout va después: primero termina el código que se está ejecutando.

Orden de salida del ejemplo 07_settimeout.js

1	A: empiezo	síncrono
2	C: sigo trabajando	síncrono
3	Pidiendo datos al «servidor»…	síncrono
4	Mientras tanto, el programa sigue	síncrono
5	D: setTimeout con 0 ms	0 ms
6	B: han pasado 1000 ms	1 s
7	Datos recibidos	2 s

Callbacks: la primera solución, y sus problemas

¿Qué es un callback?
Una función que entregas a otra para que la llame cuando termine su trabajo.

Como dejar tu teléfono
«Llámame cuando esté listo»: tú sigues con tus cosas.

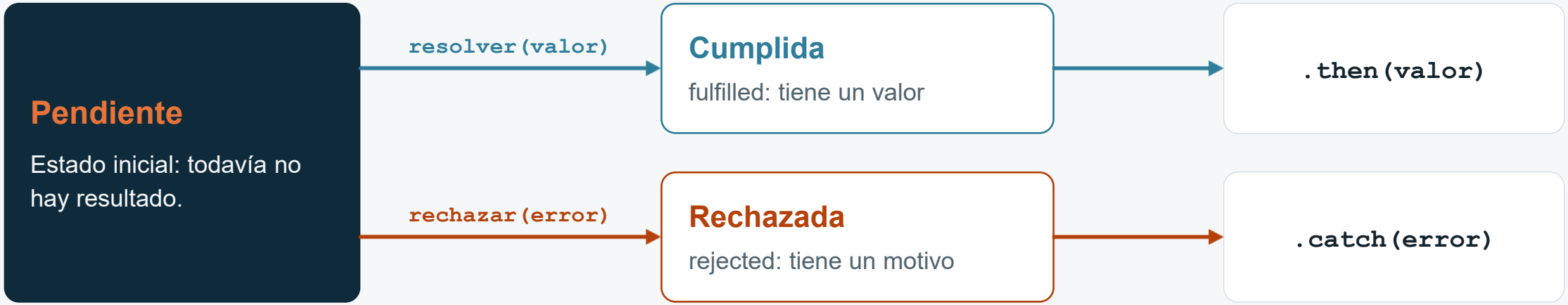
Ya lo hemos usado
La función que le pasamos a setTimeout es un callback.

El problema: callbacks dentro de callbacks

1. Pedir el usuario
2. Con él, pedir sus pedidos
3. Con ellos, pedir el pago
4. Con el pago, confirmar

Tres problemas: se anidan en pirámide, el código se lee mal y los errores son difíciles de controlar. Las promesas nacieron para resolverlos.

Una promesa: tres estados




```
promesa
  .then((mensaje) => console.log('✓ then:', mensaje))
  .catch((error) => console.log('✗ catch:', error.message))
  .finally(() => console.log('finally: siempre'));
```

Mi primera promesa

```
const todoBien = true;

const promesa = new Promise((resolver, rechazar) => {
  setTimeout(() => {
    if (todoBien) resolver('¡Datos recibidos!');
    else rechazar(new Error('Ha fallado la conexión'));
  }, 1000);
});

promesa.then(...).catch(...).finally(...);
```

La función que recibe **new Promise** se ejecuta enseguida. Con **resolver** o **rechazar** decides cómo termina.

Salida con todoBien = true

```
Estado al crearla: Promise { <pending> }
Esto sale antes de que la promesa termine
(1 segundo después)
✓ then: ¡Datos recibidos!
finally: se ejecuta siempre
```

Con todoBien = false

```
✗ catch: Ha fallado la conexión
finally: se ejecuta siempre
```

async y await: la forma cómoda de esperar

async delante de la función
Permite usar await dentro. La función pasa a devolver una promesa.

await delante de una promesa
Pausa la función (no el programa) hasta que se cumple y te da su valor.

try / catch para los errores
Si la promesa se rechaza, el catch recoge el error, igual que ya conoces.

En un módulo, await sin función
En un archivo .js con "type": "module" puedes escribir await al nivel superior.

```
const mostrarLibro = async (id) => {
  const libro = await buscarLibro(id);
  console.log(libro.titulo);
};
```

Se lee de arriba abajo, como código normal, aunque haya esperas.

async / await con try / catch / finally

buscarLibro(id) simula un servidor: tarda 0,5 s y se rechaza si el id no existe.

```
const mostrarLibro = async (id) => {
  try {
    console.log(`Buscando el libro ${id}...`);
    const libro = await buscarLibro(id);
    console.log('✓ Encontrado:', libro.titulo);
  } catch (error) {
    console.log('✗ Error:', error.message);
  } finally {
    console.log('Búsqueda terminada');
  }
};
```

await mostrarLibro(1)

```
Buscando el libro 1...
✓ Encontrado: El nombre de la rosa
Búsqueda terminada
```

await mostrarLibro(99)

```
Buscando el libro 99...
✗ Error: No existe ningún libro con id 99
Búsqueda terminada
```

ASINCRONÍA · VARIAS PROMESAS

Varias peticiones a la vez: Promise.all

Una detrás de otra (await, await, await)



Todas a la vez (Promise.all)



Tarda lo que la más lenta, no la suma de todas.

```
const libros = await Promise.all([
  buscarLibro(1),
  buscarLibro(2),
  buscarLibro(3),
]);
```

`Promise.all`
Devuelve todos los resultados juntos. Si una falla, falla todo.

`Promise.allSettled`
Espera a todas y te dice, una a una, si se cumplió o se rechazó.

Existen también Promise.race y Promise.any: de momento, solo saber que existen.

Encadenar con then

```
buscarLibro(1)
  .then((libro) => {
    console.log('1.º:', libro.titulo);
    return buscarLibro(2); // devolvemos la promesa
  })
  .then((libro) => {
    console.log('2.º:', libro.titulo);
    return buscarLibro(99); // no existe
  })
  .then((libro) => console.log('3.º:', libro.titulo))
  .catch((error) => console.log('✗ Error:', error.message))
  .finally(() => console.log('Cadena terminada'));
```

```
1.º: El nombre de la rosa
2.º: Cien años de soledad
✗ Error: No existe ningún libro con id 99
Cadena terminada
```

Cada then devuelve otra promesa
Por eso se pueden encadenar uno tras otro.

Si devuelves una promesa, la cadena espera
Con return buscarLibro(2) el siguiente then recibe su resultado.

Un solo catch al final
Recoge el error de cualquier eslabón: el tercer then ni se ejecuta.

Error típico: olvidar el return
La cadena no espera y el siguiente then recibe undefined.

ASINCRONÍA

Errores habituales con código asíncrono

Error	Qué ocurre	Solución
`const libro = buscarLibro(1);`	libro es Promise { <pending> }	**Falta el await**
`await dentro de una función normal`	SyntaxError	**Marca la función como async**
`setTimeout(saludar(), 100);`	saludar se ejecuta ya, no a los 100 ms	**Pasa la función sin paréntesis: setTimeout(saludar, 100)**
`.then(...) sin return dentro`	El siguiente then recibe undefined	**Devuelve la promesa con return**
`Promesa rechazada sin catch`	Error «sin capturar» en la consola	**Usa try/catch (o .catch)**

CIERRE

Resumen del tema

1 Errores
try/catch/finally controlan los fallos sin parar el programa; throw lanza errores propios.

2 JSON
Formato de texto para intercambiar datos; las API REST responden con JSON.

3 Sus reglas
Seis tipos de valor; comillas dobles, sin comentarios y sin coma final.

4 stringify y parse
Objeto a texto y texto a objeto. parse, siempre dentro de try/catch.

5 Copiar objetos
Igualar no copia; { ...obj } es copia superficial y structuredClone, profunda.

6 Un solo hilo
setTimeout ejecuta una función más tarde; lo escrito de seguido va antes.

7 Promesas
Pendiente, cumplida o rechazada. async/await para usarlas; Promise.all para varias.

Siguiente paso · UT 5.2
Fetch para pedir datos a un servidor real, y los fundamentos de HTTP.

CIERRE

Actividades propuestas

1 Detective de JSON
En 02_json_sintaxis.js añade tres textos con errores distintos; predice el resultado y corrígelos.

2 Ida y vuelta
Crea un objeto reserva con notasInternas; conviértelo a JSON sin esa propiedad y reconstrúyelo con parse.

3 Copias
Con el objeto ana de 04_copiar_objetos.js haz una copia superficial y otra profunda; modifica las notas y explica qué pasa.

4 Validador
Escribe esReservaValida(valor) y pruébala con cinco textos JSON: tres válidos y dos no.

5 Adivina el orden
Tres console.log y dos setTimeout (0 ms y 500 ms): predice el orden de salida y compruébalo.

6 De then a await
Reescribe la cadena de 11_encadenar_then.js con una función async y try/catch.

7 Más rápido
Pide los libros 1, 2 y 3 con tres await seguidos y con Promise.all, y mide el tiempo de cada versión.

Punto de partida
Los ejemplos de la carpeta EJEM_05.1_sencillo.

CIERRE

Práctica: los 11 ejemplos (se ejecutan con Node)

01_try_catch.js
try/catch/finally y throw

02_json_sintaxis.js
¿Es JSON válido? ✓ / ✗

03_stringify_parse.js
stringify, parse y lo que se pierde

04_copiar_objetos.js
Igualar, copia superficial y profunda

05_validar_json.js
Lectura segura y validación

06_json_modulo.js
Importar un .json como módulo

07_settimeout.js
Ejecutar más tarde y orden de salida

08_primera_promesa.js
Crear una promesa: then, catch, finally

09_async_await.js
async/await con try/catch

10_varias_promesas.js
Promise.all y Promise.allSettled

11_encadenar_then.js
Encadenar con then (código antiguo)

node 01_try_catch.js
Se ejecutan así; no hace falta servidor.